



设计模式

面向嵌入式软件开发



针对C语言及嵌入式软件开发的
伴读教程，陪伴学习设计模式这一
进阶知识点。

一、Intent（意图）

在内存受限的系统中，**用共享数据的方式表示大量小对象**，避免重复存储。

二、Motivation（动机）

系统中存在大量“看起来是多个对象”，但实际上：

- 数据结构相同
- 大部分字段相同
- 只有少量差异

典型问题：

```
struct Device {
    uint8_t type;
    uint16_t p1;
    uint16_t p2;
};

struct Device dev[500];
```

如果：

- type 只有几种
- p1/p2 取值有限

那么：

👉 绝大多数内存是在重复存同样的数据

关键切入点：

一个“对象”的数据，并不一定都需要独占

三、Applicability（适用条件）

适用于以下场景：

1. 对象数量很大

- 通道
 - UI元素
 - 字符
 - 状态节点
-

2. 数据重复率高

- 配置离散
 - 参数组合有限
 - 模式数量远小于对象数量
-

3. 可以拆分状态

对象可以拆成：

- 共享部分（不随对象变化）
 - 变化部分（运行时确定）
-

4. 不依赖对象“身份”

系统只关心：

- 数据内容
- 行为结果

而不是“这是第几个对象”

四、Structure（结构）

系统由三部分组成：

1. 共享数据池

```
const struct Config configs[];
```

- 存放所有可复用的数据
 - 通常放在 Flash
-

2. 引用（索引）

```
uint8_t index;
```

- 每个对象不再存数据
 - 只存“指向哪一份共享数据”
-

3. 使用接口

```
cfg = &configs[index];
```

- 在使用时取出共享数据
 - 外部状态通过参数传入
-

整体结构：

```
对象 = index + 外部状态  
数据 = configs[]
```

五、Participants（参与角色）

1. 共享数据（Flyweight）

```
const struct Config {  
    uint8_t type;  
    uint16_t p1;  
    uint16_t p2;  
};
```

特点：

- 不可修改
 - 可被多个对象引用
-

2. 数据池（共享集合）

```
const struct Config configs[];
```

特点：

- 唯一存储位置
- 按索引访问

3. 使用者

```
cfg = &configs[idx];
```

负责：

- 提供索引
- 提供外部参数

六、Collaborations（协作关系）

运行流程：

1. 对象持有索引（而不是完整数据）
2. 使用时通过索引访问共享数据
3. 外部状态通过函数参数传入

示意：

```
index → configs[index] → 使用 + 外部参数
```

关键点：

共享数据不包含上下文信息，所有变化由调用方提供

七、Consequences（效果）

优点

1. 大幅减少内存占用

- 从 N 份数据 → 1 份 + N 个索引

2. 用 Flash 替代 RAM

- 共享数据可放只读区

3. 数据一致性高

- 所有对象引用同一份数据

代价

1. 外部状态必须显式管理

```
draw(x, y, index);
```

- 容易遗漏或传错

2. 访问间接化

```
cfg = &configs[idx];
```

- 可读性下降

3. 设计复杂度增加

- 需要拆分状态
- 需要设计共享粒度

八、Implementation（实现要点）

1. 优先使用 const 数据

```
const struct Config configs[];
```

2. 用索引代替指针

```
uint8_t idx;
```

- 更节省空间
- 更安全

3. 外部状态不入结构体

正确：

```
draw(cfg, x, y);
```

错误：

```
struct {  
    Config cfg;  
    int x, y;  
};
```

4. 控制共享粒度

- 太细：收益小
- 太粗：灵活性差

九、Example（嵌入式示例）

```
typedef struct {  
    uint8_t mode;  
    uint16_t period;  
    uint8_t duty;  
} LedConfig;  
  
const LedConfig configs[] = {  
    {0, 0, 0},  
    {1, 500, 50},  
    {2, 1000, 30}  
};  
  
uint8_t led_idx[100];
```

使用：

```
void led_run(int id) {  
    const LedConfig* cfg = &configs[led_idx[id]];  
    run_led(cfg->mode, cfg->period, cfg->duty);  
}
```

十、总结

享元模式的核心不在“对象”，而在：

把重复数据从对象中剥离出来，转为共享存储

在嵌入式系统中，它通常表现为：

👉 查表 + const 数据 + 索引访问 + 外部参数